

EECE 7205: Fundamentals of Computer Engineering
Fall 2026
In Class Work

Student Name: Natan Blumberg
Instructor: Dr. Fatema Nafa

1 Problem 1: Bubble Sort

1.1 Understanding the Problem

- Goal: sort A into increasing order with Bubble Sort and show the array after every pass.
- Input: an integer array and its size n.
- Output: the array after each pass, the sorted array, and the number of passes, comparisons, and swaps.
- Edge cases: an already sorted array (best case), a reversed array (worst case), duplicate values, and arrays with 0 or 1 elements.

1.2 Solution Approach

Bubble Sort only ever compares and swaps neighbors. One pass walks through the unsorted part from left to right and swaps any neighbor pair where the left value is bigger. The biggest value in the unsorted part wins every comparison it takes part in, so it keeps moving right until it reaches the end of the unsorted part. It then stays there for good.

Because of this, each pass can stop one position earlier than the one before it. After pass p (counting from 0), the last p + 1 positions are final, so pass 1 makes 7 comparisons, pass 2 makes 6, and so on. The sort can also stop early. A flag called swapped is set to false at the start of each pass and to true on any swap. A pass with no swaps proves the array is sorted, so the loop ends there.

1.3 Pseudocode

Algorithm 1: Bubble Sort

BubbleSort(A, n, passes, comparisons, swaps)

Input: An array of numbers A, its size n, and counters passed by reference.

Output: A sorted in increasing order; passes, comparisons, and swaps counted.

```
1. passes <- 0, comparisons <- 0, swaps <- 0
2. for pass <- 0 to n - 2 do
3.   swapped <- False
4.   for i <- 0 to n - pass - 2 do
5.     comparisons <- comparisons + 1
6.     if A[i] > A[i + 1] then
7.       Swap(A[i], A[i + 1])
8.       swaps <- swaps + 1
9.       swapped <- True
10.    end if
11.  end for
12.  passes <- passes + 1
13.  Print "Pass ", passes, ": "
14.  PrintArray(A, n)
15.  if swapped = False then
16.    break
17.  end if
18. end for
```

1.4 Execution Snapshot

```
Test 1: original array A
Original: 34 7 23 32 5 62 14 19
Pass 1: 7 23 32 5 34 14 19 62
Pass 2: 7 23 5 32 14 19 34 62
Pass 3: 7 5 23 14 19 32 34 62
Pass 4: 5 7 14 19 23 32 34 62
Pass 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Passes: 5 Comparisons: 25 Swaps: 15

Test 2: already sorted copy of A
Original: 5 7 14 19 23 32 34 62
Pass 1: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Passes: 1 Comparisons: 7 Swaps: 0
```

1.5 Answer to Question 5: Already Sorted Copy of A

Input	Passes	Comparisons	Swaps
A: 34 7 23 32 5 62 14 19	5	25	15
Sorted copy: 5 7 14 19 23 32 34 62	1	7	0

On the sorted copy the program makes 1 pass and 7 comparisons, which is $n - 1$. Every neighbor pair is already in order, so the test $A[i] > A[i + 1]$ is never true and no swap happens. The flag swapped stays false, so the early exit ends the sort after the first pass. This is the best case, $\Theta(n)$.

2 Problem 2: Insertion Sort

2.1 Understanding the Problem

- Goal: sort A into increasing order with Insertion Sort and show the key and the array after each insertion.
- Input: an integer array and its size n.
- Output: one line for each step $i = 1$ to $n - 1$, the sorted array, and the number of comparisons and shifts.
- Edge cases: an already sorted array, a reversed array, duplicate values, and arrays with 0 or 1 elements.

2.2 Solution Approach

Insertion Sort works like sorting a hand of playing cards. The left part $A[0..i-1]$ is always sorted. At step i the next value is copied into key, and j walks left from $i - 1$. Every value bigger than key is moved one position to the right, which is a shift. When j reaches a value that is not bigger than key, or runs off the left end, key is dropped into the gap at $j + 1$.

A shift is not a swap. The key is held in its own variable, so each larger value only has to move once, one position to the right.

The inner loop needs care for counting. If the test $A[j] > key$ is written in the while condition, the natural place for the counter is inside the loop body, and then the test that fails and ends the loop is never counted. The count would always equal the number of shifts. So the loop only checks $j \geq 0$, and the value test is an if statement inside the loop with its own counter, followed by break when the test fails.

2.3 Pseudocode

Algorithm 2: Insertion Sort

InsertionSort(A, n, comparisons, shifts)

Input: An array of numbers A, its size n, and counters passed by reference.

Output: A sorted in increasing order; comparisons and shifts counted.

```
1. comparisons <- 0, shifts <- 0
2. for i <- 1 to n - 1 do
3.   key <- A[i]
4.   j <- i - 1
5.   while j >= 0 do
6.     comparisons <- comparisons + 1
7.     if A[j] > key then
8.       A[j + 1] <- A[j]
9.       shifts <- shifts + 1
10.    j <- j - 1
11.   else
12.     break
13.   end if
14. end while
15. A[j + 1] <- key
16. Print "i = ", i, ", key = ", key, ": "
17. PrintArray(A, n)
18. end for
```

2.4 Execution Snapshot

```
● Test 1: original array A
Original: 34 7 23 32 5 62 14 19
i = 1, key = 7: 7 34 23 32 5 62 14 19
i = 2, key = 23: 7 23 34 32 5 62 14 19
i = 3, key = 32: 7 23 32 34 5 62 14 19
i = 4, key = 5: 5 7 23 32 34 62 14 19
i = 5, key = 62: 5 7 23 32 34 62 14 19
i = 6, key = 14: 5 7 14 23 32 34 62 19
i = 7, key = 19: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 20 Shifts: 15

Test 2: increasing order
Original: 5 7 14 19 23 32 34 62
i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 7 Shifts: 0

Test 3: decreasing order
Original: 62 34 32 23 19 14 7 5
i = 1, key = 34: 34 62 32 23 19 14 7 5
i = 2, key = 32: 32 34 62 23 19 14 7 5
i = 3, key = 23: 23 32 34 62 19 14 7 5
i = 4, key = 19: 19 23 32 34 62 14 7 5
i = 5, key = 14: 14 19 23 32 34 62 7 5
i = 6, key = 7: 7 14 19 23 32 34 62 5
i = 7, key = 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Shifts: 28
```

2.5 Answer to Question 4: Fewest and Most Shifts

Ordering	Comparisons	Shifts
A: 34 7 23 32 5 62 14 19	20	15
Increasing: 5 7 14 19 23 32 34 62	7	0
Decreasing: 62 34 32 23 19 14 7 5	28	28

The increasing ordering gives the fewest shifts (0) and the decreasing ordering gives the most (28). Each shift fixes one inversion (a larger value before a smaller one), so the number of shifts equals the number of inversions. Increasing order has no inversions, so every key stays where it is. In decreasing order all $8 \cdot 7/2 = 28$ pairs are inversions, so every key has to move all the way to index 0.

3 Problem 3: Selection Sort

3.1 Understanding the Problem

- Goal: sort A into increasing order with Selection Sort and show the minimum found in each pass.
- Input: an integer array and its size n.
- Output: for each pass, the value and index of the minimum and the array after the swap; then the sorted array and the number of comparisons and swaps.
- Edge cases: an already sorted array (no swaps), duplicate values, and arrays with 0 or 1 elements.

3.2 Solution Approach

Selection Sort keeps a sorted part on the left and an unsorted part on the right. Pass i scans the whole unsorted part A[i..n-1] to find the index of its smallest value, then swaps that value into position i. Position i is then final, and the sorted part grows by one.

Each pass makes at most one swap, and the check $\text{minIdx} \neq i$ skips the swap when the minimum is already in place. The number of comparisons does not depend on the input at all, because every pass always scans everything that is left.

3.3 Pseudocode

Algorithm 3: Selection Sort

SelectionSort(A, n, comparisons, swaps)

Input: An array of numbers A, its size n, and counters passed by reference.

Output: A sorted in increasing order + comparisons and swaps counted.

```
1. comparisons <- 0, swaps <- 0
2. for i <- 0 to n - 2 do
3.   minIdx <- i
4.   for j <- i + 1 to n - 1 do
5.     comparisons <- comparisons + 1
6.     if A[j] < A[minIdx] then
7.       minIdx <- j
8.     end if
9.   end for
10.  if minIdx != i then
11.    Swap(A[i], A[minIdx])
12.    swaps <- swaps + 1
13.  end if
14.  Print "Pass ", i + 1, ": min = ", A[i], " at index ", minIdx, " -> "
15.  PrintArray(A, n)
16. end for
```

3.4 Execution Snapshot

```
• Test 1: original array A
Original: 34 7 23 32 5 62 14 19
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Swaps: 5
```

3.5 Answer to Question 4: Swaps Compared with Bubble Sort

Algorithm	Swaps on A
Bubble Sort (Problem 1)	15
Selection Sort (Problem 3)	5

Selection Sort makes fewer swaps: 5 against 15. Bubble Sort only swaps neighbors, and each neighbor swap fixes exactly one inversion, so it has to make one swap for each of the 15 inversions in A. Selection Sort instead moves each value straight to its final position with one swap per pass, so it makes at most $n - 1 = 7$ swaps. On A two of the passes found the minimum already in place, which leaves 5.

4 Problem 4: Quick Sort

4.1 Understanding the Problem

- Goal: sort A into increasing order with Quick Sort, using the last element of each subarray as the pivot.
- Input: an integer array and the bounds low and high of the part to sort.
- Output: the steps of the first partition, the pivot and subarray of every recursive call indented by depth, the sorted array, and the number of comparisons.
- Edge cases: a sorted or reversed array (worst case for this pivot), duplicate values, and subarrays with 0 or 1 elements.

4.2 Solution Approach

Quick Sort picks a pivot, splits the array around it, and then sorts the two sides separately. With the Lomuto partition the pivot is $A[\text{high}]$. The index i marks the end of a region of values that are less than or equal to the pivot. That region starts empty at $\text{low} - 1$. The index j scans from low to $\text{high} - 1$. Whenever $A[j] \leq \text{pivot}$, the region grows by one (i goes up) and $A[j]$ is swapped into it. At the end, swapping $A[i]$

+ 1] with the pivot puts the pivot between the two regions. Everything to its left is smaller or equal and everything to its right is larger, so the pivot is in its final sorted position.

Quick Sort then calls itself on the part left of the pivot and on the part right of it. A part with 0 or 1 elements is already sorted, which is what the test `low < high` checks.

`quickSort` takes a depth parameter, which is used for the indentation and to show the steps of the first partition only, and `partition` takes a flag called `showSteps`. Both also take a comparison counter passed by reference. A small helper, `printIndent`, prints only spaces, so all array printing still goes through `printArray`. Because `quickSort` calls itself, the comparison counter is set to 0 in `main()` before the first call instead of inside the function.

4.3 Pseudocode

Algorithm 4: Quick Sort

`QuickSort(A, lo, hi, depth, comparisons)`

Input: An array `A`, bounds `lo` and `hi`, the recursion depth, and a comparison counter passed by reference.

Output: `A[lo..hi]` sorted in increasing order.

```

1. if lo < hi then
2.   Print 2 * depth spaces, "pivot = ", A[hi], ": "
3.   PrintArray(A + lo, hi - lo + 1)
4.   showSteps <- (depth = 0)
5.   p <- Partition(A, lo, hi, showSteps, comparisons)
6.   QuickSort(A, lo, p - 1, depth + 1, comparisons)
7.   QuickSort(A, p + 1, hi, depth + 1, comparisons)
8. end if

```

`Partition(A, lo, hi, showSteps, comparisons)`

Input: An array `A`, bounds `lo` and `hi`, a flag `showSteps`, and a comparison counter passed by reference.

Output: The final index of the pivot, with smaller or equal values on its left and larger values on its right.

```

1. pivot <- A[hi]
2. i <- lo - 1
3. for j <- lo to hi - 1 do
4.   comparisons <- comparisons + 1
5.   if A[j] <= pivot then
6.     i <- i + 1
7.     Swap(A[i], A[j])
8.     if showSteps = True then
9.       Print "i = ", i, ", j = ", j, ": "
10.      PrintArray(A + lo, hi - lo + 1)
11.    end if
12.  end if
13. end for
14. Swap(A[i + 1], A[hi])
15. if showSteps = True then
16.   Print "pivot placed: "
17.   PrintArray(A + lo, hi - lo + 1)
18.   Print "pivot final position: ", i + 1
19. end if
20. return i + 1

```

4.4 Execution Snapshot

```
• Test 1: original array A
Original: 34 7 23 32 5 62 14 19
pivot = 19: 34 7 23 32 5 62 14 19
i = 0, j = 1: 7 34 23 32 5 62 14 19
i = 1, j = 4: 7 5 23 32 34 62 14 19
i = 2, j = 6: 7 5 14 32 34 62 23 19
pivot placed: 7 5 14 19 34 62 23 32
pivot final position: 3
  pivot = 14: 7 5 14
    pivot = 5: 7 5
      pivot = 32: 34 62 23 32
        pivot = 62: 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 14

Test 2: already sorted copy of A
Original: 5 7 14 19 23 32 34 62
pivot = 62: 5 7 14 19 23 32 34 62
i = 0, j = 0: 5 7 14 19 23 32 34 62
i = 1, j = 1: 5 7 14 19 23 32 34 62
i = 2, j = 2: 5 7 14 19 23 32 34 62
i = 3, j = 3: 5 7 14 19 23 32 34 62
i = 4, j = 4: 5 7 14 19 23 32 34 62
i = 5, j = 5: 5 7 14 19 23 32 34 62
i = 6, j = 6: 5 7 14 19 23 32 34 62
pivot placed: 5 7 14 19 23 32 34 62
pivot final position: 7
  pivot = 34: 5 7 14 19 23 32 34
    pivot = 32: 5 7 14 19 23 32
      pivot = 23: 5 7 14 19 23
        pivot = 19: 5 7 14 19
          pivot = 14: 5 7 14
            pivot = 7: 5 7
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28
```

4.5 Answer to Question 5: Already Sorted Copy of A

Input	Comparisons
A: 34 7 23 32 5 62 14 19	14
Sorted copy: 5 7 14 19 23 32 34 62	28

On the sorted copy the program makes 28 comparisons, twice as many as on A. When the array is sorted, the last element of every subarray is its largest value, so partition leaves the pivot at the end. The split is always $n - 1$ elements on the left and 0 on the right, so each call only removes the pivot. The comparisons are $7 + 6 + 5 + 4 + 3 + 2 + 1 = 28 = n(n - 1)/2$, which is the worst case, $\Theta(n^2)$.

One way to avoid this is the median-of-three pivot. Before partitioning, swap the middle value of $A[lo]$, $A[mid]$, and $A[hi]$ into $A[hi]$. On a sorted array this picks the true middle value, so every split is even and the sort takes $\Theta(n \log n)$.

5 Problem 5: Merge Sort

5.1 Understanding the Problem

- Goal: sort A into increasing order with Merge Sort, using $mid = (low + high) / 2$.
- Input: an integer array and the bounds low and high of the part to sort.
- Output: every split subarray, every merged subarray, the sorted array, the total number of comparisons, and the number made in the final merge.
- Edge cases: sorted and reversed arrays, duplicate values, and parts with 0 or 1 elements.

5.2 Solution Approach

Merge Sort splits the array at $mid = (low + high) / 2$ and keeps splitting until every part has one element. A single element is already sorted. It then merges pairs of sorted parts back together, from the bottom of the tree to the top.

The merge copies the two halves into temporary arrays L and R. It compares the front value of L with the front value of R and writes the smaller one back into A. When one half runs out, the rest of the other half is copied over with no more comparisons, since it is already sorted. The test $L[i] \leq R[j]$ takes from L on a tie, which keeps equal values in their original order.

mergeSort gets three extra parameters: depth, a comparison counter, and finalMerge. The top call has depth 0, so the merge made at depth 0 is the final merge. The program records the counter just before that merge and subtracts it afterward. Because mergeSort calls itself, both counters are set to 0 in main() before the first call.

5.3 Pseudocode

Algorithm 5: Merge Sort

MergeSort(A, lo, hi, depth, comparisons, finalMerge)

Input: An array A, bounds lo and hi, the recursion depth, and counters passed by reference.

Output: $A[lo..hi]$ sorted in increasing order.

1. if $lo < hi$ then
2. Print "split: "
3. PrintArray(A + lo, hi - lo + 1)
4. $mid \leftarrow \text{floor}((lo + hi) / 2)$
5. MergeSort(A, lo, mid, depth + 1, comparisons, finalMerge)
6. MergeSort(A, mid + 1, hi, depth + 1, comparisons, finalMerge)
7. before $\leftarrow$ comparisons
8. Merge(A, lo, mid, hi, comparisons)

```

9.   if depth = 0 then
10.     finalMerge <- comparisons - before
11.   end if
12. end if

Merge(A, lo, mid, hi, comparisons)
Input: An array A whose parts A[lo..mid] and A[mid + 1..hi] are sorted,
       and a comparison counter passed by reference.
Output: A[lo..hi] sorted in increasing order.
1. L <- A[lo..mid], R <- A[mid + 1..hi]
2. i <- 0, j <- 0, k <- lo
3. while i < SizeOf(L) and j < SizeOf(R) do
4.   comparisons <- comparisons + 1
5.   if L[i] <= R[j] then
6.     A[k] <- L[i], i <- i + 1
7.   else
8.     A[k] <- R[j], j <- j + 1
9.   end if
10.  k <- k + 1
11. end while
12. while i < SizeOf(L) do
13.  A[k] <- L[i], i <- i + 1, k <- k + 1
14. end while
15. while j < SizeOf(R) do
16.  A[k] <- R[j], j <- j + 1, k <- k + 1
17. end while
18. Print "merged: "
19. PrintArray(A + lo, hi - lo + 1)

```

5.4 Execution Snapshot

```
Test 1: original array A
Original: 34 7 23 32 5 62 14 19
split: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
merged: 7 34
split: 23 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
merged: 5 62
split: 14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 17 (final merge: 7)

Test 2: sorted copy of A
Original: 5 7 14 19 23 32 34 62
split: 5 7 14 19 23 32 34 62
split: 5 7 14 19
split: 5 7
merged: 5 7
split: 14 19
merged: 14 19
merged: 5 7 14 19
split: 23 32 34 62
split: 23 32
merged: 23 32
split: 34 62
merged: 34 62
merged: 23 32 34 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)

Test 3: reversed copy of A
Original: 62 34 32 23 19 14 7 5
split: 62 34 32 23 19 14 7 5
split: 62 34 32 23
split: 62 34
merged: 34 62
split: 32 23
merged: 23 32
merged: 23 32 34 62
split: 19 14 7 5
split: 19 14
merged: 14 19
split: 7 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)
```

5.5 Answer to Question 5: Sorted and Reversed Copies

Input	Comparisons	Final merge
A: 34 7 23 32 5 62 14 19	17	7
Sorted copy: 5 7 14 19 23 32 34 62	12	4
Reversed copy: 62 34 32 23 19 14 7 5	12	4

Merge Sort makes between 12 and 17 comparisons on the three inputs, while Bubble Sort goes from 7 to 28 and Quick Sort goes from 14 to 28. Merge Sort always splits at the middle, whatever the values are, so it makes the same splits and merges for every input. Only the work inside each merge changes, and a merge of m values always makes between $m / 2$ and $m - 1$ comparisons. On the sorted and reversed copies one half runs out first in every merge, so every merge makes the smallest number, $m / 2$.

Extra memory: the temporary arrays L and R hold copies of the two halves, so Merge Sort needs $\Theta(n)$ extra memory.

6 Problem 6: Heap Sort

6.1 Understanding the Problem

- Goal: sort A into increasing order with Heap Sort, using a binary max-heap stored inside the array.
- Input: an integer array and its size n .
- Output: the heap after it is built, the heap part and the sorted part after every pass of the sorting phase, the sorted array, and the number of swaps inside `siftDown`.
- Edge cases: duplicate values (stability), and arrays with 0 or 1 elements.

6.2 Solution Approach

The array is treated as a binary tree: the root is at index 0 and the children of index k are at $2k + 1$ and $2k + 2$. In a max-heap every parent is at least as large as its children, so the largest value is always at the root.

`siftDown` fixes one position. It compares the value at root with its children, and if the larger child is bigger, it swaps the two and keeps going from the child's position. It stops when the value is at least as large as both children, or has no children.

The sort has two phases. The build phase calls `siftDown` on every parent, from the last one (index $n/2 - 1$) back to the root. Leaves have no children, so they are already heaps. The sorting phase then repeats one step for `end = n - 1` down to 1: swap the root (the largest value left) into $A[\text{end}]$, shrink the heap to the first `end` elements, and sift the new root down. The swap of the root into $A[\text{end}]$ happens in `heapSort`, not in `siftDown`, so it is not counted.

`siftDown` takes the array, the index of the root to fix, and the heap size, plus a swap counter passed by reference. `printArray` ends every line, so the heap part and the sorted part print on two lines. The word sorted is indented so the two lines line up.

6.3 Pseudocode

Algorithm 6: Heap Sort

`HeapSort(A, n, swaps)`

Input: An array of numbers A , its size n , and a swap counter passed by reference.

Output: A sorted in increasing order; swaps inside `SiftDown` counted.

```
1. swaps <- 0
2. for i <- floor(n / 2) - 1 down to 0 do
3.   SiftDown(A, i, n, swaps)
4. end for
5. Print "Heap: "
```

```

6. PrintArray(A, n)
7. for end <- n - 1 down to 1 do
8.   Swap(A[0], A[end])
9.   SiftDown(A, 0, end, swaps)
10.  Print "end = ", end, ": heap: "
11.  PrintArray(A, end)
12.  Print "sorted: "
13.  PrintArray(A + end, n - end)
14. end for

```

SiftDown(A, root, size, swaps)

Input: An array A, the index root to fix, the heap size, and a swap counter passed by reference.

Output: The subtree at root is a max-heap again.

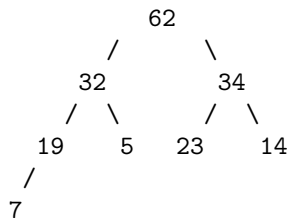
```

1. largest <- root, l <- 2 * root + 1, r <- 2 * root + 2
2. if l < size and A[l] > A[largest] then
3.   largest <- l
4. end if
5. if r < size and A[r] > A[largest] then
6.   largest <- r
7. end if
8. if largest != root then
9.   Swap(A[root], A[largest])
10.  swaps <- swaps + 1
11.  SiftDown(A, largest, size, swaps)
12. end if

```

6.4 Heap After the Build Phase

The heap after the build phase is 62 32 34 19 5 23 14 7. As a binary tree:



6.5 Execution Snapshot

```
• Test 1: original array A
Original: 34 7 23 32 5 62 14 19
Heap: 62 32 34 19 5 23 14 7
end = 7: heap: 34 32 23 19 5 7 14
        sorted: 62
end = 6: heap: 32 19 23 14 5 7
        sorted: 34 62
end = 5: heap: 23 19 7 14 5
        sorted: 32 34 62
end = 4: heap: 19 14 7 5
        sorted: 23 32 34 62
end = 3: heap: 14 5 7
        sorted: 19 23 32 34 62
end = 2: heap: 7 5
        sorted: 14 19 23 32 34 62
end = 1: heap: 5
        sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 12
```

6.6 Answer to Question 5: Is Heap Sort Stable?

No. A sort is stable if equal values come out in the same order they went in. Heap Sort can change that order. Take the small array $\{5a, 5b, 3\}$, where 5a and 5b are equal and the letters only show their original order (5a first).

Step	What happens	Heap part	Sorted part
Start		5a 5b 3	
Build: siftDown(0)	5b and 3 are not larger than 5a, no swap	5a 5b 3	
end = 2	swap root 5a into A[2]	3 5b	5a
	siftDown(0): 5b is larger than 3, swap	5b 3	5a
end = 1	swap root 5b into A[1]	3	5b 5a

The result is 3, 5b, 5a: the two 5s came out in reverse order, so Heap Sort is not stable. The swap of the root into A[end] moves a value from the front of the array to the back in one jump, past any equal values.

7 Problem 7: Comparing the Six Algorithms

7.1 Understanding the Problem

- Goal: check the theoretical running times of the six sorts with measurements.
- Inputs: A, a sorted copy of A, a reversed copy of A, and random arrays of 1,000, 5,000, and 10,000 integers between 1 and 100,000.
- Output: a table of comparisons on the three small inputs, the time and comparisons of each sort on each random array, and the first 10 values of each sorted random array.
- Edge cases: Quick Sort's worst case on sorted and reversed input.

7.2 Solution Approach

The six sorting functions are copied from Problems 1 to 6 with every print statement removed. Each one now takes a single comparison counter, which main() resets to 0 before every call. Heap Sort is the one real change: Problem 6 counted swaps, but this table needs comparisons, so siftDown now counts every test of a child against the largest value so far. A helper function, runSort, takes a number from 0 to 5 and calls the matching sort, so main() can loop over all six. For the random arrays, the values are made once for each size and then copied into a fresh array before every sort, so all six sorts get exactly the same input. The random numbers come from the random library.

7.3 Pseudocode

```

SiftDown(A, root, size, comparisons)
1. largest ← root, l ← 2 * root + 1, r ← 2 * root + 2
2. if l < size then
3.   comparisons ← comparisons + 1
4.   if A[l] > A[largest] then
5.     largest ← l
6.   end if
7. end if
8. if r < size then
9.   comparisons ← comparisons + 1
10.  if A[r] > A[largest] then
11.    largest ← r
12.  end if
13. end if
14. if largest ≠ root then
15.   Swap(A[root], A[largest])
16.   SiftDown(A, largest, size, comparisons)
17. end if

```

Main

```

1. A <- {34, 7, 23, 32, 5, 62, 14, 19}
2. n <- SizeOf(A)
3. S <- {5, 7, 14, 19, 23, 32, 34, 62}
4. for k <- 0 to n - 1 do
5.   R[k] <- S[n - 1 - k]
6. end for
7. for each of the six sorts do
8.   for each input X in {A, S, R} do
9.     B <- copy of X
10.    comparisons <- 0
11.    Sort B with this algorithm, counting comparisons
12.    Print comparisons
13.   end for
14. end for
15. for each size m in {1000, 5000, 10000} do
16.   for k <- 0 to m - 1 do
17.     data[k] <- Random(1, 100000)
18.   end for
19.   for each of the six sorts do
20.     B <- copy of data
21.     comparisons <- 0
22.     start <- Clock()
23.     Sort B with this algorithm, counting comparisons
24.     stop <- Clock()
25.     Print algorithm name, stop - start, comparisons
26.     PrintArray(B, 10)
27.   end for
28. end for

```

(SiftDown is written out again because it now counts comparisons instead of swaps. The other five sorts are the same as in Problems 1 to 5 with the printing removed.)

7.4 Results: Comparisons on A, Sorted A, and Reversed A

Algorithm	A	Sorted copy	Reversed copy
Bubble Sort	25	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	14	28	28
Merge Sort	17	12	12
Heap Sort	26	27	24

7.5 Results: Running Time on Random Arrays

Algorithm	n = 1,000	n = 5,000	n = 10,000
Bubble Sort	1.0615 ms	21.661 ms	83.8601 ms
Insertion Sort	0.4101 ms	7.6478 ms	32.3519 ms
Selection Sort	0.562 ms	11.4228 ms	48.0026 ms
Quick Sort	0.121 ms	0.3624 ms	0.7267 ms
Merge Sort	0.2304 ms	1.1069 ms	2.5407 ms
Heap Sort	0.0862 ms	0.4605 ms	1.0918 ms

7.6 Execution Snapshot

```
Part 1: comparisons on A, sorted A, and reversed A
Algorithm      A      Sorted Reversed
Bubble         25      7      28
Insertion      20      7      28
Selection      28      28     28
Quick          14      28     28
Merge          17      12     12
Heap           26      27     24

Part 2: running time on random arrays

n = 1000
Bubble    time: 1.0615 ms  comparisons: 498680
  first 10: 52 121 170 195 203 365 374 602 638 778
Insertion  time: 0.4101 ms  comparisons: 240785
  first 10: 52 121 170 195 203 365 374 602 638 778
Selection  time: 0.562 ms   comparisons: 499500
  first 10: 52 121 170 195 203 365 374 602 638 778
Quick      time: 0.121 ms   comparisons: 11124
  first 10: 52 121 170 195 203 365 374 602 638 778
Merge      time: 0.2304 ms  comparisons: 8709
  first 10: 52 121 170 195 203 365 374 602 638 778
Heap       time: 0.0862 ms  comparisons: 16815
  first 10: 52 121 170 195 203 365 374 602 638 778

n = 5000
Bubble    time: 21.661 ms  comparisons: 12492144
  first 10: 20 31 35 50 78 107 112 143 175 186
Insertion  time: 7.6478 ms  comparisons: 6170382
  first 10: 20 31 35 50 78 107 112 143 175 186
Selection  time: 11.4228 ms  comparisons: 12497500
  first 10: 20 31 35 50 78 107 112 143 175 186
Quick      time: 0.3624 ms  comparisons: 70402
  first 10: 20 31 35 50 78 107 112 143 175 186
Merge      time: 1.1069 ms  comparisons: 55247
  first 10: 20 31 35 50 78 107 112 143 175 186
Heap       time: 0.4605 ms  comparisons: 107652
  first 10: 20 31 35 50 78 107 112 143 175 186

n = 10000
Bubble    time: 83.8601 ms  comparisons: 49987619
  first 10: 17 42 46 46 70 84 98 104 131 132
Insertion  time: 32.3519 ms  comparisons: 24870369
  first 10: 17 42 46 46 70 84 98 104 131 132
Selection  time: 48.0026 ms  comparisons: 49995000
  first 10: 17 42 46 46 70 84 98 104 131 132
Quick      time: 0.7267 ms  comparisons: 149626
  first 10: 17 42 46 46 70 84 98 104 131 132
Merge      time: 2.5407 ms  comparisons: 120387
  first 10: 17 42 46 46 70 84 98 104 131 132
Heap       time: 1.0918 ms  comparisons: 235507
  first 10: 17 42 46 46 70 84 98 104 131 132
```

7.7 Answers to Question 4

Which algorithms grow like n^2 and which grow like $n \log n$? Bubble Sort, Insertion Sort, and Selection Sort grow like n^2 . When n doubles from 5,000 to 10,000, their times grow by a factor of 3.9 to 4.2. Quick Sort, Merge Sort, and Heap Sort grow like $n \log n$. Their times grow by a factor of 2.0 to 2.4.

Do the measurements agree with the theory? Yes. Doubling n should multiply an n^2 algorithm's time by $2^2 = 4$, and an $n \log n$ algorithm's time by $2 \cdot \log(10000)/\log(5000) \approx 2.16$. The measured ratios are close to both numbers. The small differences come from timer noise, since some of these runs take less than 1 ms.

Which algorithm for a large random array? Quick Sort. It was the fastest at 5,000 and 10,000 values, it sorts in place, and its average case is $\Theta(n \log n)$. Its worst case only happens with bad pivots, which random data does not produce.

Which algorithm for an array that is already almost sorted? Insertion Sort. Its work is about n plus the number of inversions, and an almost sorted array has very few inversions, so it runs in close to linear time. On the sorted copy of A it made only 7 comparisons. Quick Sort with the last element as the pivot is the wrong choice here, because almost sorted input is close to its worst case.

AI Use Disclaimer I Used AI to:

- Question me about faults in my pseudocode.
- Teach me some concepts in C++.
- Identify errors and bugs in my code.
- Assist in writing the LaTeX for this assignment. (Tables, equations, etc.)
- Explain several sorting methods since we have not learned them completely in class.

AI did NOT:

- Write my code (Pseudocode or C++)
- Do any thinking or problem solving that impacted methods on solving these problems.